

Day 55: Search Product and Return None

Day 55: Search Product and Return None

1. Day Number

Day 55

2. Topic Name

Search method and None

Today you will learn how to search through a list of objects and:

- return the matching object when found
 - return None when nothing matches
-

3. Connection

Previously, you removed products from a cart by searching for a product name.

Today, instead of removing the product, you will **return the product object itself**.

The idea is:

```
Search product
  |
  v
Found?
 /  \
Yes  No
 |    |
return return
object None
```

This pattern is extremely common in Python applications.

4. Important Topics

Today's main concepts are:

- for loop
 - return
 - None
 - list of objects
 - object attributes
 - search logic
 - method/function
-

5. Foundational Notes

What does searching mean?

Suppose your program contains three products:

```
Laptop  
Mouse  
Keyboard
```

The user searches for:

```
Mouse
```

Your program checks each product one by one.

```
Laptop == Mouse?    No  
Mouse == Mouse?     Yes
```

Once the product is found, the program can immediately return it.

What is return?

return sends a value back from a function or method.

Simple example:

```
def get_number():  
    return 10
```

Then:

```
number = get_number()
```

Now:

```
number = 10
```

What is None?

None represents **no value / nothing found / no result**.

For example:

```
result = None
```

It does not mean:

```
0
```

It does not mean:

```
""
```

It means:

```
there is currently no object/value here
```

Why return None when searching?

Suppose you search for:

```
Phone
```

but your product list contains only:

```
Laptop  
Mouse  
Keyboard
```

There is no product object to return.

So returning:

```
None
```

clearly tells the caller:

```
| No matching product was found.
```

List of objects

Instead of storing simple strings:

```
products = ["Laptop", "Mouse"]
```

you may have objects:

```
Product object  
  name = Laptop  
  price = 50000  
  
Product object  
  name = Mouse  
  price = 500
```

The list contains the **objects themselves**.

Conceptually:

```
products  
|  
+--> Product("Laptop")  
|
```

```
---> Product("Mouse")
|
---> Product("Keyboard")
```

When searching, you check an attribute such as:

```
product.name
```

6. Easy Example

Before using Product objects, understand the same idea with numbers.

Suppose:

```
numbers = [10, 20, 30]
```

You want to search for:

```
20
```

The logic is:

```
Start loop

10 == 20?
No

20 == 20?
Yes

Return 20
```

If searching for:

```
50
```

then:

```
10 == 50? No
20 == 50? No
30 == 50? No

Loop finished

Return None
```

The important pattern is:

```
FOR each item
  IF item matches
    RETURN item
```

RETURN None

Notice that return None happens **after the loop finishes**.

7. Problem Statement

Create a product list.

Each product should be represented using a Product object.

Write a function or method that:

1. accepts a product name to search for
2. loops through the product list
3. compares the requested name with each product's name
4. returns the matching Product object if found
5. returns None if no matching product exists

For example:

Products:

Laptop
Mouse
Keyboard

Search:

Mouse

Result:

Mouse product object

Search:

Phone

Result:

None

8. Concepts Used

Class

Represents a product.

Conceptually:

```
Product
├── name
├── price
└── stock
```

Object

An individual product created from the class.

For example:

```
Laptop Product object
```

List of objects

Store multiple Product objects together.

```
products
├── laptop_object
├── mouse_object
└── keyboard_object
```

Loop

Visit every product.

```
for product in products:
```

Attribute access

Check the product's name.

```
product.name
```

if

Compare the current product against the requested name.

return

Immediately send the matching product back.

None

Represent the situation where no product was found.

9. Thought Process

Think about the problem in this order.

Step 1: What am I searching?

A product name.

For example:

Mouse

Step 2: Where are the products?

Inside a list.

Conceptually:

```
products = [  
    Laptop object,  
    Mouse object,  
    Keyboard object  
]
```

Step 3: How can I inspect every product?

Use a loop.

```
for every product in products
```

Step 4: What should I compare?

Compare:

```
current product name
```

with:

```
requested product name
```

Step 5: What happens if they match?

Return the current Product object.

```
Laptop -> no  
Mouse -> yes  
  
RETURN Mouse object
```

The search can stop immediately.

Step 6: What if the loop finishes without finding anything?

Return:

```
None
```

Overall flow:

```
Search "Mouse"  
  |  
  v  
+-----+  
| Laptop |
```

```

+-----+
|       |
|       |
| no match |
|       |
|       |
|       |
+-----+
| Mouse |
|       |
+-----+
|       |
|       |
| match  |
|       |
|       |
|       |
+-----+
Return Mouse object

```

Not-found flow:

```

Search "Phone"
|
v
Laptop -> no
|
Mouse -> no
|
Keyboard -> no
|
Loop finished
|
v
Return None

```

10. Pseudocode

OOP-style pseudocode:

```

CREATE Product class

CREATE Cart or ProductManager class

STORE Product objects in a list

DEFINE search_product(product_name)

    FOR each product in product list

        IF product.name equals product_name
            RETURN product

    RETURN None

```

Notice this important structure:


```
FOR ...
  IF ...
    RETURN product

RETURN None
```

The final `RETURN None` should usually be **outside the loop**.

11. Suggested Solving Approach

Preferred: OOP-Based Approach

Since you are already working with `Product` and `Cart`, continue using OOP.

A possible design is:

```
Product
├── name
├── price
└── stock

Cart
├── items
├── add_product()
├── remove_product()
├── calculate_total()
└── search_product()
```

Conceptually:

```
cart.search_product("Mouse")
```

may return:

```
<Product object for Mouse>
```

while:

```
cart.search_product("Phone")
```

may return:

```
None
```

This approach fits naturally with your previous cart exercises.

Functional Approach

A standalone function is also acceptable.

Conceptually:

```
search_product(products, product_name)
```

The function receives:

```
product list  
+  
name being searched
```

and returns:

```
Product object
```

or:

```
None
```

For this learning sequence, the **OOP approach is preferred**.

12. Easy Edge Cases

Edge Case 1: Product Not Found

Products:

```
Laptop  
Mouse  
Keyboard
```

Search:

```
Phone
```

No product matches.

Expected result:

```
None
```

Edge Case 2: Empty Product List

Suppose:

```
products = []
```

Search:

```
Laptop
```

The loop has nothing to inspect.

Therefore, the result should be:

None

This is useful because you don't need special complicated logic for an empty list.

Conceptually:

```
Empty list
|
FOR loop runs 0 times
|
RETURN None
```

13. Expected Input

Example products:

```
Product 1:
Name: Laptop
Price: 50000
Stock: 5

Product 2:
Name: Mouse
Price: 500
Stock: 20

Product 3:
Name: Keyboard
Price: 1500
Stock: 10
```

Search input:

Enter product name: Mouse

Another possible input:

Enter product name: Phone

14. Expected Output

When the product exists:

```
Enter product name: Mouse
Product found: Mouse
```

You may also display other attributes:

```
Product found
Name: Mouse
```

```
Price: 500
Stock: 20
```

Internally, however, your search function/method should return the **Product object**, not just print the name.

For a missing product:

```
Enter product name: Phone
Product not found
```

Internally:

```
None
```

A useful calling pattern is conceptually:

```
result = search for product

IF result is None
    print product not found
ELSE
    print result's details
```

15. Hint Only

Think about these three lines of logic:

```
for product in ...:
```

Then compare:

```
product.name
```

with the searched name.

If they match:

```
return ...
```

After the **entire loop finishes**, think about:

```
return None
```

The key structure is:

```
loop
|
|   +-- match? --> return Product object
|
loop ends
|
```

```
        v
    return None
```

Important beginner point: don't put `return None` inside the loop too early. Otherwise, your method might check only the first product and stop before searching the rest.